

# LABORATORY OBSERVATION

**Course:** Full Stack Web Development

**Course Code:** 23CM4121

**Branch:** CSM

**Section:** B

**Task No.:** 1(b)

**Student Name:** M. Niharika

**Roll No.:** A24126552097

---

## 1. TITLE

**Draw 2D Graphics Using HTML5 Canvas and Style a Webpage Using CSS3**

---

## 2. AIM

To draw basic 2D shapes and text on an **HTML5 canvas** using JavaScript, and to style a complete webpage — header, navigation bar, content cards, and footer — using **CSS3** properties such as gradients, shadows, and hover effects.

---

## 3. OBJECTIVE

The objectives of this experiment are:

- To understand the basic structure and purpose of the `<canvas>` element.
  - To obtain a 2D drawing context using `getContext("2d")`.
  - To draw a rectangle using `fillRect()`.
  - To draw a circle using `arc()` and `fill()`.
  - To draw a straight line using `moveTo()`, `lineTo()`, and `stroke()`.
  - To render text on a canvas using `fillText()`.
  - To style a page header using CSS gradients and text-shadow.
  - To create a navigation bar with hover effects using CSS pseudo-classes.
  - To lay out content cards using Flexbox.
  - To apply box-shadow, border-radius, and other box-model properties for visual depth.
- 

## 4. THEORY

The **HTML5 <canvas> element** is simply an empty rectangular region on the page. It has no drawing ability of its own — every shape, line, or piece of text visible inside it is painted using JavaScript, through an object called the drawing context.

CSS3 is used separately to style the rest of the page — the header, navigation, cards, and footer — using properties for colour, spacing, shadows, and layout.

The key APIs and properties used in this program are:

- **getContext("2d")** returns the drawing context object that provides all the draw functions used below.
- **fillStyle** sets the colour used by the next filled shape or piece of text.
- **fillRect(x, y, w, h)** draws a filled rectangle at position (x, y) with the given width and height.
- **beginPath()** and **arc()** start a new shape and then draw a circular arc; a full sweep (0 to  $2\pi$ ) makes a complete circle.
- **moveTo()** and **lineTo()** move the pen without drawing, then draw a straight line to the next point; **stroke()** renders it.
- **fillText(text, x, y)** draws text on the canvas, with (x, y) marking the bottom-left of the text baseline.
- **linear-gradient()** blends two colours smoothly across an element's background, used on the header.
- **:hover** applies a style only while the mouse pointer is over an element — used on links, buttons, and cards.
- **box-shadow** and **border-radius** give the cards and header a soft, rounded, elevated appearance.

The basic flow of the page is:

**HTML loads the canvas element → JavaScript gets its 2D context → Draw commands paint shapes onto the canvas → CSS3 rules style the rest of the page**

---

## 5. ALGORITHM / PROCEDURE

1. Create two files: index.html and style.css, and link the CSS file inside the HTML head.
2. Add a <canvas> element with a fixed width and height inside a section.
3. In a <script> tag, get a reference to the canvas element using its id.
4. Call **getContext("2d")** on the canvas to obtain the drawing context.
5. Set **fillStyle** to green and draw a rectangle using **fillRect()**.
6. Begin a new path and draw a full circle using **arc()**, then fill it blue.
7. Begin a new path, use **moveTo()** and **lineTo()** to define a straight line, and render it using **stroke()**.
8. Set the font and **fillStyle**, then draw a name on the canvas using **fillText()**.
9. Add a semantic header, a navigation bar with links, three content cards, and a footer below the canvas section.
10. In style.css, reset default margin/padding and set the body as a flex column for a sticky footer.
11. Style the header with a linear-gradient background and a text-shadow.
12. Style the navigation links with spacing and a **:hover** colour change.
13. Lay the cards out using Flexbox, with **border-radius** and **box-shadow** for depth.
14. Style the buttons with padding, rounded corners, and a hover background change.
15. Save the files and open index.html in a browser to verify the canvas drawing and page styling.

---

## 6. PROGRAM

index.html

```
<!DOCTYPE html>
<html lang="en">
```

```

<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>My Styled Webpage</title>
<link rel="stylesheet" href="style.css">
</head>
<body>

<!-- ===== PART A: CANVAS GRAPHICS ===== -->
<div class="canvas-section">
  <h1>Canvas Graphics</h1>
  <canvas id="myCanvas" width="400" height="250"></canvas>
</div>
<!-- <canvas> is just an empty rectangular box on the page.
      Nothing is drawn yet - it has NO drawing ability of its own.
      Everything visible inside it is drawn using JavaScript. -->

<script>
  const canvas = document.getElementById("myCanvas");
  const ctx = canvas.getContext("2d");
  // getContext("2d") gives us the "drawing tool" (called the context)
  // that has all the draw functions like fillRect, arc, etc.
  // Without this line, none of the drawing commands below would work.

  // ----- RECTANGLE -----
  ctx.fillStyle = "green";
  // fillStyle sets the COLOR used for the next shape drawn
  ctx.fillRect(20, 20, 100, 60);
  // fillRect(x, y, width, height)
  // x,y = top-left corner position of the rectangle
  // width,height = size of the rectangle

  // ----- CIRCLE -----
  ctx.beginPath();
  // beginPath() starts a NEW shape - without this, the circle
  // would connect to the previous shape's path by mistake
  ctx.arc(220, 60, 40, 0, Math.PI * 2);
  // arc(x, y, radius, startAngle, endAngle)
  // 0 to Math.PI*2 means "go all the way around" = a full circle
  ctx.fillStyle = "blue";
  ctx.fill();
  // fill() actually paints the shape with the current fillStyle

  // ----- LINE -----
  ctx.beginPath();
  ctx.moveTo(20, 150);
  // moveTo(x, y) moves the "pen" to a starting point WITHOUT drawing
  ctx.lineTo(300, 150);
  // lineTo(x, y) draws a straight line from the last point to this one
  ctx.stroke();
  // stroke() actually renders the line (moveTo/lineTo alone draw nothing)

  // ----- TEXT (Name) -----
  ctx.font = "20px Arial";
  ctx.fillStyle = "black";

```

```

    ctx.fillText("Niharika", 20, 200);
    // fillText(text, x, y) - x,y is the position of the BOTTOM-LEFT
    // of the text, not the top-left (a common beginner mistake)
</script>

<!-- ===== PART B: CSS STYLING DEMO ===== -->
<header class="site-header">My Styled Webpage</header>
<!-- <header> = semantic tag for the top banner section of the page -->

<nav class="navbar">
    <!-- <nav> = semantic tag telling the browser "this block is for navigation" -->
    <a href="#">Home</a>
    <a href="#">About</a>
    <a href="#">Contact</a>
</nav>

<div class="cards">
    <!-- A plain <div> is used here since "cards" is just a layout
         container, not a meaningful content type on its own -->
    <div class="card">
        <h3>HTML</h3>
        <p>HTML (HyperText Markup Language) provides the basic structure and content of a webpage
using elements like headings, paragraphs, images, and links.</p>
        <button>Learn More</button>
    </div>

    <div class="card">
        <h3>CSS</h3>
        <p>CSS (Cascading Style Sheets) styles web pages by adding colors, fonts, layouts, spacing,
animations, and responsive designs for a better appearance.</p>
        <button>Learn More</button>
    </div>

    <div class="card">
        <h3>JavaScript</h3>
        <p>JavaScript adds interactivity to web pages by handling events, updating content
dynamically, validating forms, and creating animations.</p>
        <button>Learn More</button>
    </div>
</div>

<footer class="site-footer">&copy; 2026 My Website</footer>
</body>
</html>

```

## style.css

```

html, body{
    height:100%;
    margin:0;
    padding:0;
}

body{
    display:flex;

```

```

    flex-direction:column;
    min-height:100vh;
    background:#f5f5f5;
}/* ===== HEADER ===== */
.site-header {
    background: linear-gradient(to right, #345979, #00b8c2);
    /* linear-gradient() blends two colors smoothly into one another,
       instead of the header being one flat, boring color */
    color: white;
    font-family: 'Segoe UI', sans-serif;
    /* font-family sets the TYPEFACE — a different, more modern
       font than the browser's plain default */
    font-size: 28px;
    text-align: center;
    padding: 20px;
    /* padding here pushes the text away from the header's own edges,
       so the text isn't jammed right against the border */
    text-shadow: 2px 2px 4px rgba(0,0,0,0.4);
    /* text-shadow draws a soft shadow BEHIND the letters,
       making white text stand out more against the gradient */
}

.canvas-section{
    width:90%;
    text-align:center;
    margin:30px auto;
}

canvas{
    width:100%;
    max-width:500px;
    height:350px;
    border:2px solid black;
}

/* ===== NAVIGATION BAR ===== */
.navbar {
    background-color: #f0f0f0;
    padding: 10px;
    text-align: center;
}

.navbar a {
    color: #333;
    margin: 0 15px;
    /* margin here creates gaps BETWEEN each link, since margin is
       space OUTSIDE an element, pushing neighbours away */
    text-decoration: none;
    font-size: 18px;
    font-family: Georgia, serif;
    /* a different font family used here vs. the header,
       to satisfy "different font families and sizes" */
}

.navbar a:hover {
    color: #ff6600;
}

```

```

/* :hover = a pseudo-class, styles applied ONLY while the
   mouse pointer is over the link - this is the "hover effect" */
}

/* ===== CONTENT CARDS ===== */
.cards {
  display: flex;
  justify-content: center;
  gap: 20px;
  margin: 30px;
}

.card {
  background-color: #f4f4f4;
  border: 1px solid #ccc;
  /* border is part of the box's own outline - takes up layout space */
  border-radius: 12px;
  /* border-radius rounds off the sharp corners of the box */
  padding: 30px;
  /* padding here creates space INSIDE the card, between its
     border and the heading/paragraph/button inside it */
  width: 300px;
  height: 200px;
  box-shadow: 3px 3px 10px rgba(0,0,0,0.2);
  /* box-shadow adds a soft shadow around the whole card,
     giving it a slight "lifted off the page" depth effect */
  text-align: center;
}

/* ===== BUTTON ===== */
button {
  padding: 10px 20px;
  border: none;
  border-radius: 6px;
  background-color: #4facfe;
  color: white;
  cursor: pointer;
  /* cursor:pointer changes the mouse icon to a hand,
     signalling "this is clickable" */
}

button:hover {
  background-color: #0077cc;
  /* hover effect: background color changes when the mouse is over it */
}

/* ===== FOOTER ===== */
.site-footer {
  width: 100%;
  background-color: #222;
  color: white;
  text-align: center;
  padding: 15px;
  margin-top: auto;
  /* margin-top pushes the footer away from whatever content
     is above it, keeping the layout from feeling cramped */
}

```

```
}
```

---

## 7. CODE EXPLANATION

| Code                                                                  | Explanation                                                                                                       |
|-----------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------|
| <code>&lt;canvas id="myCanvas"&gt;</code>                             | An empty drawing surface; JavaScript is required to paint anything inside it.                                     |
| <code>getContext("2d")</code>                                         | Returns the 2D drawing context object with all draw functions ( <code>fillRect</code> , <code>arc</code> , etc.). |
| <code>fillStyle</code>                                                | Sets the colour used by the next filled shape or piece of text.                                                   |
| <code>fillRect(x, y, w, h)</code>                                     | Draws a filled rectangle at (x, y) with the given width and height.                                               |
| <code>beginPath()</code> , <code>arc()</code> , <code>fill()</code>   | Start a new shape, define a circular arc, then paint it — used to draw the circle.                                |
| <code>moveTo()</code> , <code>lineTo()</code> , <code>stroke()</code> | Move the pen, draw a straight line to a point, then render the line.                                              |
| <code>fillText(text, x, y)</code>                                     | Draws text on the canvas, with (x, y) as the bottom-left of the text.                                             |
| <code>linear-gradient(to right, ...)</code>                           | Blends two colours smoothly across the header's background.                                                       |
| <code>text-shadow</code>                                              | Draws a soft shadow behind text so it stands out against the gradient.                                            |
| <code>:hover</code>                                                   | Applies a style only while the mouse pointer is over a link, button, or card.                                     |
| <code>display: flex; gap</code>                                       | Arranges the three content cards in a row with even spacing.                                                      |
| <code>box-shadow</code> , <code>border-radius</code>                  | Give the cards a soft, rounded, elevated ('lifted') appearance.                                                   |
| <code>margin-top: auto</code>                                         | Pushes the footer to the bottom of the page in the flex-column body layout.                                       |

---

## 8. TEST CASES AND EXECUTION DATA

The page was opened in a browser and each drawn shape and styled element was checked individually; the results were recorded below.

| Test Case | Action                              | Expected Result                                                    | Actual Result                           | Status |
|-----------|-------------------------------------|--------------------------------------------------------------------|-----------------------------------------|--------|
| TC-01     | Open index.html in a browser        | Canvas displays a green rectangle, a blue circle, a line, and text | All shapes and text displayed correctly | Pass   |
| TC-02     | Check rectangle position and colour | Green rectangle appears at the top-left of the canvas              | Rectangle displayed as expected         | Pass   |
| TC-03     | Check circle position and colour    | Blue circle appears to the right of the rectangle                  | Circle displayed as expected            | Pass   |
| TC-04     | Check the drawn line                | A straight horizontal line appears below the shapes                | Line displayed as expected              | Pass   |
| TC-05     | Check the text on canvas            | "Niharika" appears near the bottom of the canvas                   | Text displayed as expected              | Pass   |
| TC-06     | Hover over a nav link               | Link colour changes to orange                                      | Hover effect displayed                  | Pass   |
| TC-07     | Hover over a 'Learn More' button    | Button background changes to a darker blue                         | Hover effect displayed                  | Pass   |

### Additional Validation Test

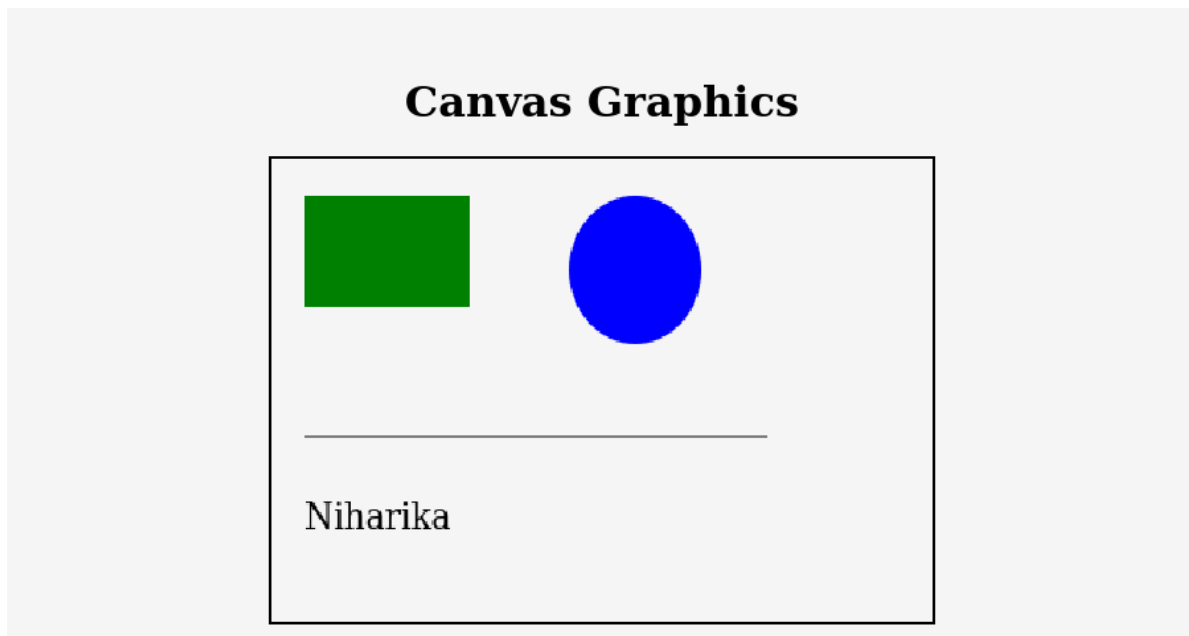
| Test Case | Input/Action                                              | Expected Result                                    | Status                                            |
|-----------|-----------------------------------------------------------|----------------------------------------------------|---------------------------------------------------|
| TC-08     | Check that the footer stays at the bottom on a short page | Footer sticks to the bottom of the viewport        | Pass                                              |
| TC-09     | Check the 'Learn More' button on each card                | Button should be fully visible below the card text | Fail – button is hidden behind the footer because |



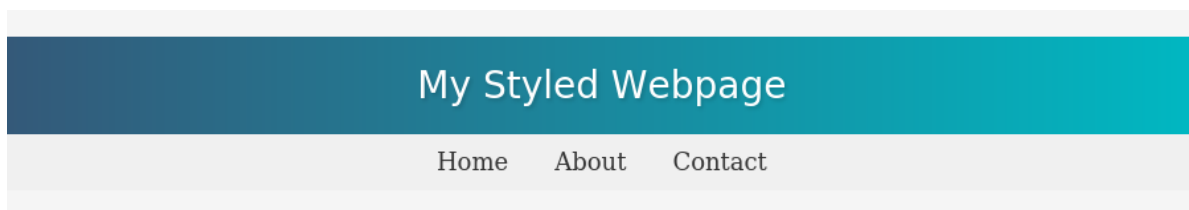
|  |  |  |                                                                                     |
|--|--|--|-------------------------------------------------------------------------------------|
|  |  |  | the<br>card's<br>fixed<br>height<br>(200px)<br>is<br>smaller<br>than its<br>content |
|--|--|--|-------------------------------------------------------------------------------------|

## 9. OUTPUT

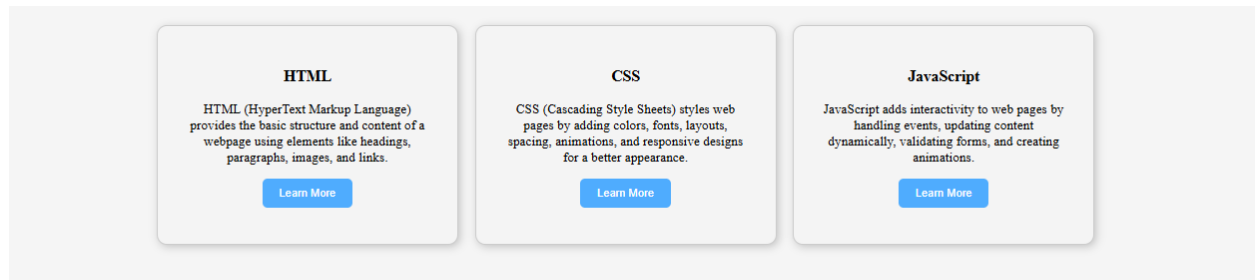
### Output 1: Canvas Graphics – Rectangle, Circle, Line, and Text



### Output 2: Styled Header (Gradient) and Navigation Bar



### Output 3: Content Cards and Footer



---

## 10. RESULT

Thus, canvas graphics — a rectangle, a circle, a line, and text — were successfully drawn using JavaScript, and a complete webpage was styled using CSS3 with a gradient header, hover effects, Flexbox cards, and shadows. During testing it was observed that the ‘Learn More’ buttons on the content cards are hidden behind the footer, because each card's fixed height (200px) is not enough for its content; removing the fixed height or increasing it would resolve this.